Hackathon Technical Solution Report

Field	Details
Problem ID	`PS-05`
Problem Title	`Detect Cars and Identify Their Make, Model and Year`
Team Name	`Quantum Minds` (Team ID `T70`)
Team Members	Kukkadapu Sai Havish (1RUA24CSE0212) · Harshitha Narasimha (1RUA24CSE0158) · Harshitha R (1RUA24CSE0159) · Dhyey Abhay Ghoda (1RUA24CSE0134)
GitHub Repository	https://github.com/ksaihavish/vericar. Test-set evaluation ran once at commit `42290c2` (recorded in `results/test_run.json`); the deployed app runs the latest `main`.
Live Demo / Deployment	https://vericar-vbmbegg4p63baw6pfc8pcz.streamlit.app/ (Streamlit Community Cloud)

---

1. Problem Understanding & Solution Strategy

Core Technical Challenge:

The main challenge is to detect cars in images and accurately identify their make, model and manufacturing year. Different car models may have very similar shapes and appearances, making classification difficult. Variations in lighting, camera angles, image quality, background conditions and partial obstruction can also affect detection and recognition accuracy.

The system should detect multiple cars in an image, identify each vehicle separately and provide its predicted make, model, year and confidence score.

We framed the problem around a concrete user: a parking or fleet officer checking whether a car matches its **registration** (cloned plates are deliberately put on look-alike cars). The question becomes "does this photo match the claimed make → model → year?", which turns the fine-grained difficulty into the core of the task. Stanford Cars stops at 2012, so the system must also recognise when a car is **outside its knowledge** instead of forcing one of the 196 labels.

- **Proposed Methodology:** A fine-tuned YOLO11n detector localises every car in the photo (each confident detection, up to six, is classified and numbered separately). A fine-tuned ConvNeXt-Tiny classifies the crop into the 196 Stanford Cars classes, and classical computer vision runs around it: a quality gate, adaptive contrast/gamma enhancement, a contour-based fallback ROI and handcrafted features. The probabilities are temperature-calibrated and summed into make, model and year levels, so a claimed registration is verified **level by level** with strict, validation-fitted thresholds (MATCH / PARTIAL / INSPECT / MISMATCH), and two independent unknown-car checks produce "not in records" for cars the model was never trained on. We chose this adaptive, calibrated design over a static top-1 classifier: a confident wrong label is the costly error in this use case, and the system has to know when to refuse.

---

2. Technical Architecture & Implementation

> *Note: Do not paste raw code files in this document. Reference repository modules and explain system design.*

The architecture diagram is in `docs/architecture.md`; the single entry point is `VeriCar.analyze(image, claim)` in `vericar/pipeline.py`.

2.1 Classical Algorithmic Baseline

- **Algorithms Utilized** (`vericar/preprocessing.py`, `vericar/classical.py`):
- Preprocessing: gray-world colour constancy, CLAHE on the Lab L-channel, gamma correction.
- Edges and ROI: auto-Canny edge detection, then morphological closing, external contours, convex hull and a largest-contour region of interest. This is a learning-free detector.
- Features, 3,913 values per crop:
    - HOG (shape)
    - multi-scale uniform LBP and GLCM statistics (texture)
    - HSV histograms and Lab colour moments (colour)
    - geometric descriptors: aspect ratio, extent, solidity, circularity, log Hu moments and a 4×4 edge-density grid
- Classifier: StandardScaler → PCA-256 (whitened) → RBF-SVM with Platt probabilities. It beat a 500-tree random forest on validation, 21.3% vs 9.3% top-1.
- **Adaptive Selection Logic:**
- The Canny thresholds are computed from the image's **median intensity** (0.67× and 1.33× the median), so there are no hand-tuned values.
- The morphological closing kernel scales with the image (3% of the longest side).
- Enhancement runs only when it is needed: gamma is applied if mean brightness (HSV-V) < 60, with the exponent computed to move the mean towards 110; CLAHE is applied if the L-channel standard deviation < 28 or after gamma. Well-exposed photos pass through untouched.
- A **quality gate** returns RETAKE when brightness < 35, the variance of the Laplacian < 35 (blur), or the crop is smaller than 48 px. These thresholds sit below the 1st percentile of the training images (`data/summary.json`), so they only fire on clearly bad input.

2.2 Deep Learning Pipeline

- **Model Architecture:**
- **Detector (`vericar/detector.py`):** YOLO11n, pretrained on COCO and fine-tuned as a single-class "car" detector. Deployed as ONNX with our own letterbox, decoding and NMS, so the live app needs no PyTorch.
- **Classifier (`vericar/classifier.py`):** ConvNeXt-Tiny (ImageNet-22k → 1k) at 384×384, with dropout 0.4 and drop-path 0.1. It is a strong transfer backbone for fine-grained recognition, it trains within a 6 GB laptop GPU in mixed precision, and it runs on a free CPU host (≈120 ms per crop in ONNX).
- **Loss Functions & Training:**
- **Detector:** standard YOLO losses (CIoU box, DFL, BCE classification), 12 epochs, AdamW with a cosine schedule.
- **Classifier:** cross-entropy with label smoothing 0.1, AdamW (weight decay 0.05), trained in two stages:
    1. backbone frozen, linear head only (3 epochs)
    2. last two ConvNeXt stages plus head, with **discriminative learning rates** (head 5e-4 > stage 3 1.2e-4 > stage 2 4.8e-5), warm-up and cosine decay
- **Overfitting guards:** hard epoch cap, early stopping on validation loss, best checkpoint by validation top-1, and a stop if the train−validation accuracy gap exceeds 10 points. The final gap was +4.6 points; best validation top-1 was 94.1% at stage-2 epoch 14.

- **Augmentation** keeps make, model and year intact: flips, box-margin and position jitter, brightness/contrast/saturation jitter, light random erasing.
- **Data:** the Stanford Cars official training set split 85/15, stratified by class with seed 42 (6,922 train / 1,222 validation). The official test set (8,041 images, labels verified against the original annotation file) was used exactly once.

2.3 Hybrid Integration & Adaptive Mechanism

- **Fusion Strategy** (`vericar/hybrid.py`): classical priors guide the network at four points.
- The **YOLO crop** (classical contour ROI as fallback) is what the CNN sees.
- The classical **quality gate and adaptive enhancement** decide whether the CNN sees the raw or the enhanced crop, or whether the system refuses (RETAKE).
- **Late log-linear fusion** with the classical SVM probabilities, where the weight w is fitted on validation negative log-likelihood. It came out as w = 0, and we report that honestly.
- **Temperature scaling** (T = 0.60, validation) calibrates the probabilities used by every downstream decision.
- **Unknown-car checks** (either one flags the car):
- cosine kNN distance from the CNN embedding to the training embeddings, with the threshold at the 95th percentile on validation
- an open-source zero-shot model (OpenCLIP ViT-B/32 LAION-2B, image encoder in ONNX INT8; `vericar/openset.py`) comparing the crop with names of the 196 known classes plus 343 post-2012 models, with the threshold set so 3% of validation cars are flagged. It was added after the test run and is evaluated separately; it can only send a car to manual check, never clear one.
- **Decision Logic:** class probabilities are summed into make, model family and year. The system balances **missed clones** (clearing a look-alike) against **false alarms** (sending a legitimate car to a manual check). Since a missed clone is the costlier error, it is tuned strict:
- **MATCH** requires the claimed class to be the **top-1** prediction *and* calibrated P ≥ 0.83. That is the lowest threshold that clears at most 2% of *hard clones* (the most similar other class) on validation.
- **PARTIAL** means the model family has P ≥ 0.80 but the year can't be separated.
- **MISMATCH** means P(claimed model family) < 0.01, which keeps legitimate mismatches ≤ 1% on validation.
- Everything else is **INSPECT**.
- In identify mode, predictions below a confidence threshold (0.49, set so that answered predictions are ≥ 95% accurate on validation) show the top 3 instead of one answer.

---

3. Experimental Benchmarks & Evaluation

3.1 Quantitative Benchmarks

Official Stanford Cars test set, 8,041 images, evaluated once (full tables: `docs/eval_P5_T70.md`). The template's PSNR, SSIM and LPIPS columns don't apply to recognition, so they are replaced with the P5 metrics.

Approach / Model Variant	Top-1 ↑	Top-5 ↑	Detection mAP@0.5 / mean IoU ↑	Calibration ECE ↓	Latency (ms / image, CPU)
Classical Baseline (contour ROI + handcrafted features + SVM)	5.7%	15.3%	0.640 / 0.654	0.004	≈ 16
Deep Learning Only (YOLO11n crop → ConvNeXt-Tiny)	93.4%	99.2%	0.990 / 0.953	0.145	≈ 205
Proposed Hybrid System	93.4%	99.2%	0.990 / 0.953	**0.012**	≈ 288 (end to end, one car; +≈ 200 per extra car)

Further results on the same test set: - **Detection:** the COCO-pretrained YOLO11n scored mAP@0.5 0.855 and mAP@0.5:0.95 0.782; after fine-tuning, 0.990 and **0.933**. - **Errors by level (hybrid):** make accuracy 97.8%, model-family accuracy 94.0%. Of the 533 errors, 57% are the same make with a different model, 35% another make, and 8% the same model with a different year. - **Claim verification (strict): 98.3%** of hard clones and 100% of easy clones are caught, at a 17.1% false-alarm rate on legitimate claims. - **Optimisation (validation, 2 CPU threads):** - ONNX FP32: 112 MB, 346 ms, 93.5% - FP16: 56 MB, 400 ms, 93.5% - INT8 dynamic: 28 MB, 348 ms, 92.75% - INT8 static: 28.7 MB, **85 ms**, 85.0% (not shipped) - Shipped: FP32 - **Robustness (1,000-image subset, hybrid top-1):** clean 93.6%, dark 90.4%, bright 92.9%, low contrast 93.1%, occlusion 10/25/40% 91.7 / 85.2 / 75.8%, ¼ resolution 79.2%, ⅛ resolution 38.9%, blur σ2/σ4 84.2 / 50.9%, noise 91.4%.

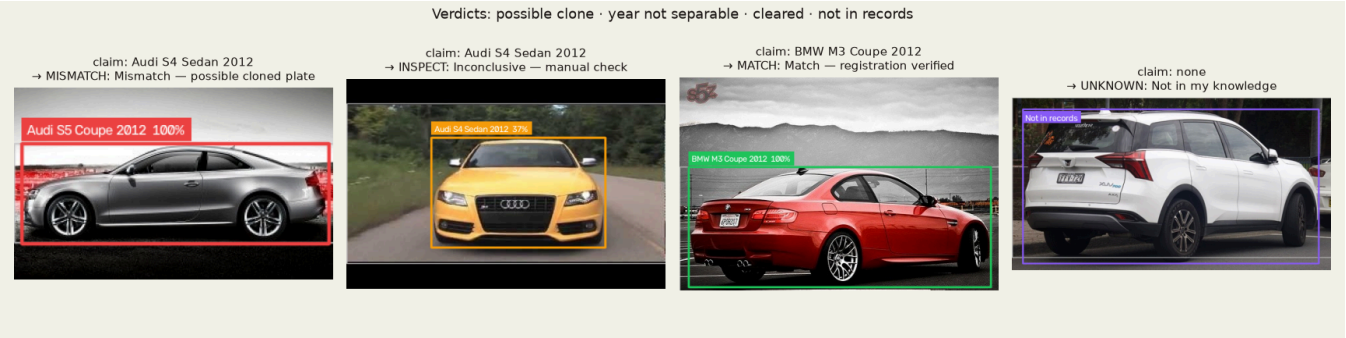
- **Analysis:** Deep learning dominates recognition. The classical pipeline is limited by its imprecise contour ROI (21.3% on clean ground-truth crops in validation, 5.7% end to end), and its validation-fitted fusion weight came out as 0, so the hybrid's top-1 equals deep-only. The hybrid's measurable gain is **trustworthiness**: calibration error drops 12× (0.145 → 0.012), which the strict verifier depends on (without calibration, false alarms rise from 17.1% to 42.6%), plus the quality gate and unknown-car checks. That costs about 80 ms over deep-only, still far inside the 10-second budget.

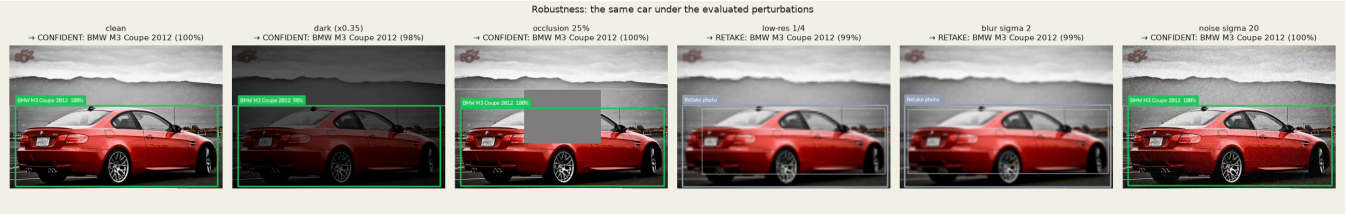
3.2 Visual Outputs & Intermediate Representations

Figures in the repository (regenerate with `python scripts/make_figures.py`):

1. **Intermediate states** (`results/figures/pipeline_stages.png`): YOLO detection → auto-Canny edges → contour, hull and classical ROI → HOG → LBP → class activation map. The CAM concentrates on the grille and badge, the cues that separate the S4 years.
2. **Verdicts** (`results/figures/verdict_examples.png`):
3. an S5 photographed against an S4 registration → **possible clone**
4. an S4 whose year can't be separated → **manual check**
5. a matching M3 → **cleared**
6. a 2021+ Mahindra XUV700 → **not in records**
7. **Varying conditions** (`results/figures/robustness_examples.png`): one car under darkness, occlusion, quarter resolution, blur and noise. The label survives all of them, but the quality gate asks for a retake at low resolution and blur.
8. **Failure panels** (`results/failures/*.png`): detection, classifier crop, Grad-CAM and reference images for each analysed failure.
9. **Live:** every intermediate image is shown for any uploaded photo on the app's **Pipeline steps** tab, and the **Classical vs deep vs hybrid** tab compares the three methods on that photo.







3.3 Ablation Study

Test set, one component removed at a time from the full hybrid:

Variant	Top-1	ECE	Hard-clone catch	False alarms
Full hybrid	93.4%	0.012	98.3%	17.1%
− adaptive preprocessing	93.4%	0.011	98.3%	17.0%
− YOLO ROI crop (full frame)	92.2%	0.014	98.4%	24.0%
− classical feature fusion (w = 0)	93.4%	0.012	98.3%	17.1%
− temperature calibration	93.4%	0.145	99.3%	**42.6%**
− quality gate & unknown check	93.4%	0.012	98.2%	13.9%

Classical feature families (validation, SVM top-1 21.3%): removing HOG drops it to **8.3%**, texture to 20.0%, colour to 20.2%, geometry to 20.9%. Shape carries the classical model, and paint colour says almost nothing about a car model. Under the dark perturbation, adaptive enhancement *hurt* (90.4% vs 93.5% without it), because the CNN's brightness augmentation already copes with darkness. We report it as a negative result.

---

4. Failure Modes & Limitations

- **Observed Failure Cases** (`results/failures/`, analysed in `docs/eval_P5_T70.md`):
- **Badge-engineered twins:** Dodge Durango 2007 predicted as Chrysler Aspen 2009 at 100%. They share the body and differ only in the grille, which a side view hides.

- **Year look-alikes:** Honda Odyssey 2007 predicted as 2012. The photo shows the 2011+ body style, so this is possibly a dataset label error.
- **Unusual viewpoint and wrong ROI:** an aerial shot of an Aston Martin. The detector box covers mostly road, and the squashed crop is unseen in training.
- **Low light plus occlusion:** Rolls-Royce Phantom vs Ghost at night, with a person blocking the side profile.
- **Truncation:** an Audi convertible cut off at both bumpers, predicted as a BMW 1 Series at 22%.
- **Root Cause & Mitigation:**
- Cases 1, 2 and 4 are inherent fine-grained ambiguity: the discriminative parts (grille, lamps, badge) are missing from the view. The mitigation is structural. Low confidence and level-wise checks turn these into PARTIAL/INSPECT, never MATCH. Requesting a front view, or adding a part-attention head, would help further.
- Case 3 comes from a train/test viewpoint gap. Viewpoint augmentation (perspective warps) and a minimum aspect-ratio check on detector boxes would mitigate it.
- Case 5 could be addressed by detecting frame-edge truncation in the quality gate.
- **Limitations:**
- **Dataset coverage:** Stanford Cars ends in 2012 and covers US-market models. Cars outside it are flagged "not in records" (25 of 25 modern probe photos), but they can't be verified. The open-set fallback names the exact model correctly only 36% of the time (top-3: 52%), and Gemini's optional AI identification is labelled unverified.
- **Strictness:** the strict policy sends 17% of legitimate cars to a manual check.
- **Over-eager quality gate:** it requests retakes for low-resolution and blurred photos that would often still be classified correctly.
- **Multi-car photos:** every confident vehicle (up to 6) is boxed, numbered and identified with its own verdict, and a registration claim is checked against the main (highest-scoring) car. Stanford Cars labels one car per image, so the multi-car path is demonstrated (app demo "Two cars", `scripts/run_inference.py`) but has no ground truth to score it against.

---

5. Deployment & Quickstart Guide

Quickstart Commands

```
 # 1. Clone the repository and install dependencies
git clone https://github.com/ksaihavish/vericar.git
cd vericar
pip install -r requirements.txt

# 2. Run inference on a sample directory (models are downloaded and SHA-256-verified on first use)
python scripts/run_inference.py --input_dir ./assets/demo/ --output_dir ./results/inference/

# Optional: verify every photo against a claimed registration
python scripts/run_inference.py --input_dir ./assets/demo/ --output_dir ./results/inference/ --
claim "Audi S4 Sedan 2012"

# 3. Launch the web app (or use run.ps1 / run.sh / start.bat, which do all of the above)
streamlit run app/streamlit_app.py
```

The batch script writes one annotated image per input and a `predictions.jsonl` containing the box, verdict, make/model/year, confidence, unknown-car reasons, each method's top-1 and the stage timings. To

reproduce training and evaluation, use `requirements-train.txt` and `python scripts/run_all.py` (steps and logs are listed in `README.md`).

Deployment Format

- **Streamlit** web app on **Streamlit Community Cloud** (CPU), built from this repository.
- **ONNX Runtime** for all three networks: YOLO11n detector, ConvNeXt-Tiny classifier with in-graph class activation map, and the INT8 OpenCLIP image encoder. scikit-learn runs the classical SVM. **No PyTorch at runtime:** about 0.55 GB of RAM and about 0.3 s per photo.
- Model files ship as a GitHub release (`models-v1`) and are downloaded and checksum-verified on first start.
- A `Dockerfile` (CPU image) and one-command launchers (`run.ps1`, `run.sh`, `start.bat`) cover local or self-hosted use.
- The optional `GEMINI_API_KEY` (officer note, AI identification for unknown cars) is a host secret and never committed. The app works fully without it.